

1. Capture versi python, direktori kerja, path file dan mounting drive atau folder tempat area file/data



```
[31]: 1 from google.colab import drive
      2
      3 drive.mount('/content/drive')

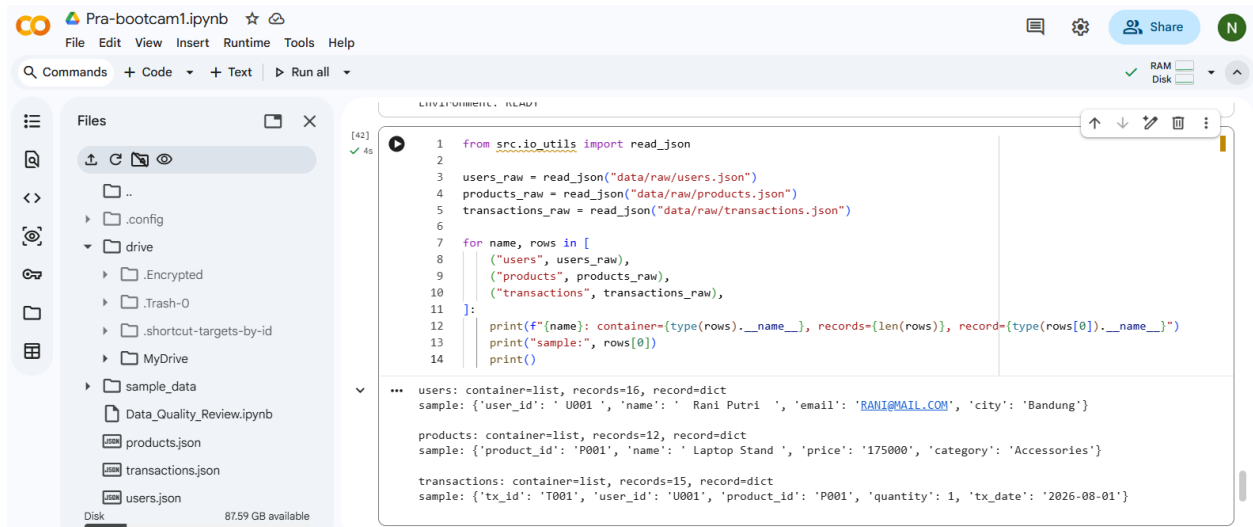
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[48]: 1 cd /content/drive/MyDrive/FSB_Data_Quality_Instructor_Pack
      ... /content/drive/MyDrive/FSB_Data_Quality_Instructor_Pack

[41]: 1 from pathlib import Path
      2 import sys, json, importlib, subprocess
      3
      4 print("Python:", sys.version.split()[0])
      5 print("Working directory:", Path.cwd())
      6 assert Path("src/cleaners.py").exists(), "Buka notebook dari root project"
      7 print("Environment: READY")
      8

Python: 3.13.15
Working directory: /content/drive/MyDrive/FSB_Data_Quality_Instructor_Pack
Environment: READY
```

2. Showing data dari file json dengan buat fungsi dengan looping For..In



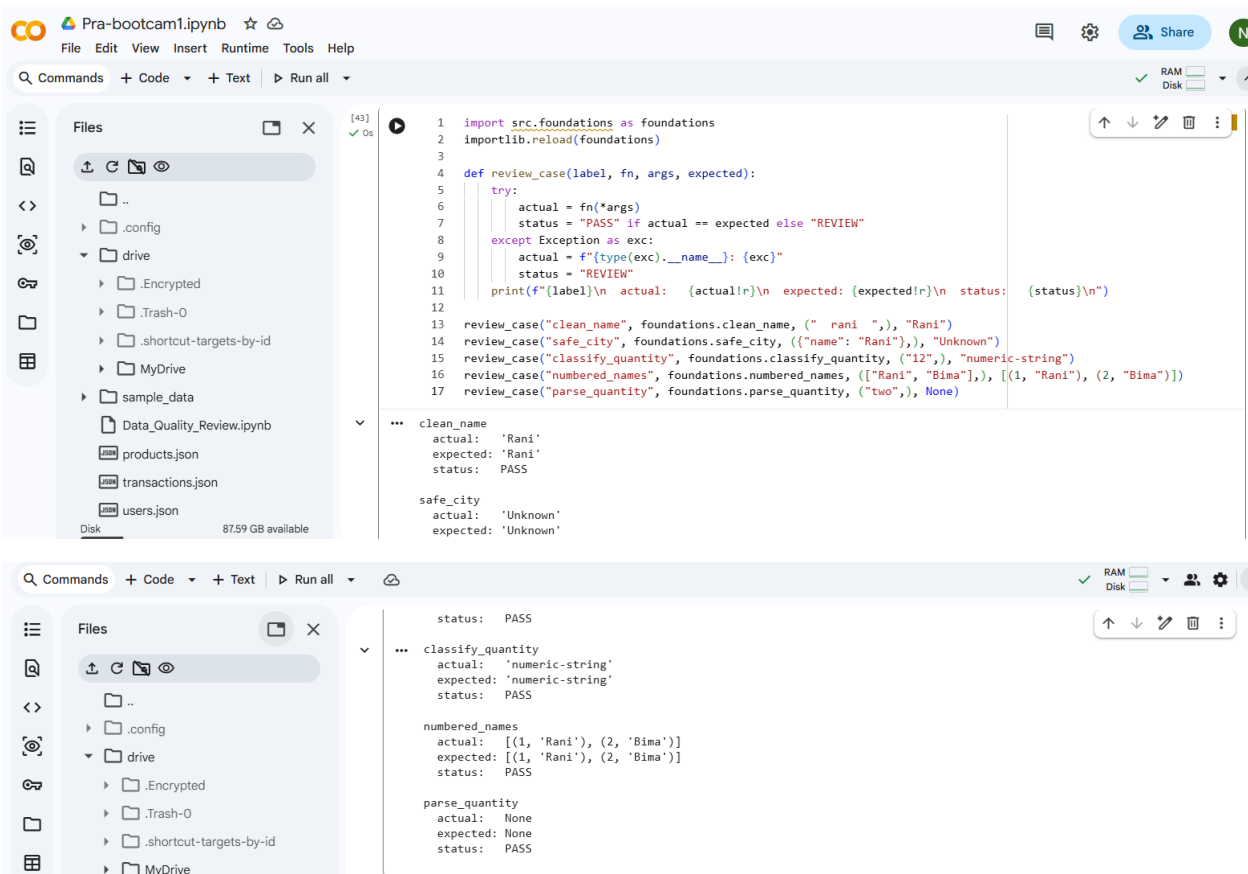
```
[42]: 1 from src.io_utils import read_json
      2
      3 users_raw = read_json("data/raw/users.json")
      4 products_raw = read_json("data/raw/products.json")
      5 transactions_raw = read_json("data/raw/transactions.json")
      6
      7 for name, rows in [
      8     ("users", users_raw),
      9     ("products", products_raw),
     10    ("transactions", transactions_raw),
     11 ]:
     12     print(f"{name}: container={type(rows).__name__}, records={len(rows)}, record={type(rows[0]).__name__}")
     13     print("sample:", rows[0])
     14     print()

... users: container=list, records=16, record=dict
sample: {'user_id': 'U001', 'name': 'Rani Putri', 'email': 'RANI@MAIL.COM', 'city': 'Bandung'}

products: container=list, records=12, record=dict
sample: {'product_id': 'P001', 'name': 'Laptop Stand', 'price': '175000', 'category': 'Accessories'}

transactions: container=list, records=15, record=dict
sample: {'tx_id': 'T001', 'user_id': 'U001', 'product_id': 'P001', 'quantity': 1, 'tx_date': '2026-08-01'}
```

3. Identify missing value dari data user membandingkan antara akutual vs expected data.



The screenshot shows a Jupyter Notebook titled 'Pra-bootcam1.ipynb'. The code defines a `review_case` function that compares actual data with expected data and returns a status ('PASS' or 'REVIEW'). The function is then used to review various data points from a dataset.

```
1 import src_foundations as foundations
2 importlib.reload(foundations)
3
4 def review_case(label, fn, args, expected):
5     try:
6         actual = fn(*args)
7         status = "PASS" if actual == expected else "REVIEW"
8     except Exception as exc:
9         actual = f"{type(exc).__name__}: {exc}"
10        status = "REVIEW"
11    print(f"{label}\n actual: {actual}\n expected: {expected}\n status: {status}\n")
12
13 review_case("clean_name", foundations.clean_name, (" rani ",), "Rani")
14 review_case("safe_city", foundations.safe_city, ({ "name": "Rani"},), "Unknown")
15 review_case("classify_quantity", foundations.classify_quantity, ("12",), "numeric-string")
16 review_case("numbered_names", foundations.numbered_names, ([ "Rani", "Bima"],), [(1, "Rani"), (2, "Bima")])
17 review_case("parse_quantity", foundations.parse_quantity, ("two",), None)
```

The output shows the results of the reviews:

```
clean_name
actual: 'Rani'
expected: 'Rani'
status: PASS

safe_city
actual: 'Unknown'
expected: 'Unknown'
```

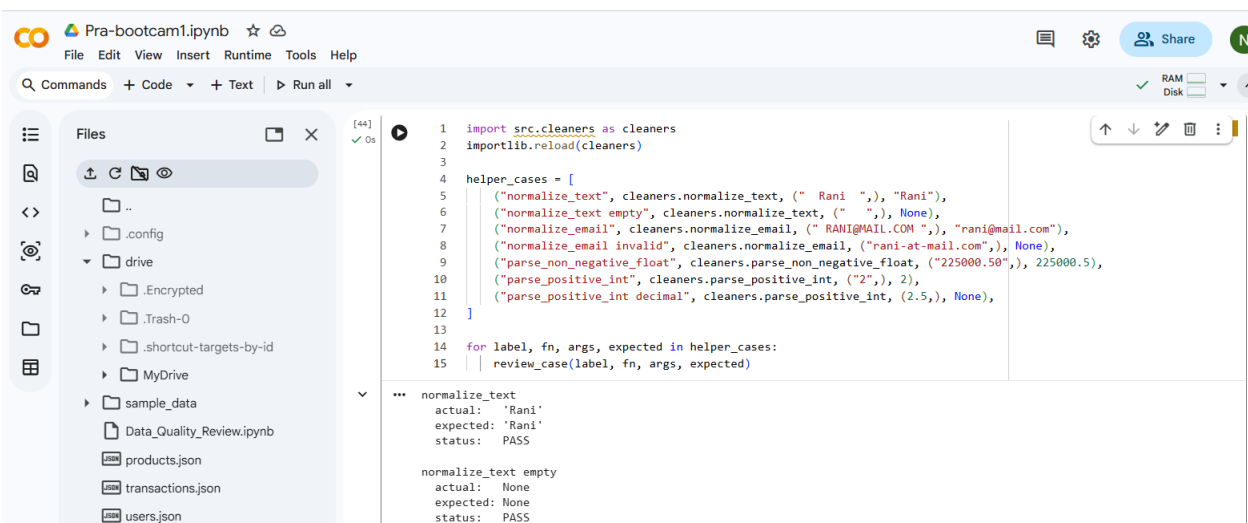
```
status: PASS

classify_quantity
actual: 'numeric-string'
expected: 'numeric-string'
status: PASS

numbered_names
actual: [(1, 'Rani'), (2, 'Bima')]
expected: [(1, 'Rani'), (2, 'Bima')]
status: PASS

parse_quantity
actual: None
expected: None
status: PASS
```

4. Replace missing value user agar sesuai standar data quality (kontrak)



The screenshot shows a Jupyter Notebook titled 'Pra-bootcam1.ipynb'. The code defines a `helper_cases` list containing various data cleaning rules using the `cleaners` module. These rules are then applied to the data using the `review_case` function.

```
1 import src_cleaners as cleaners
2 importlib.reload(cleaners)
3
4 helper_cases = [
5     ("normalize_text", cleaners.normalize_text, (" Rani ",), "Rani"),
6     ("normalize_text empty", cleaners.normalize_text, (" ",), None),
7     ("normalize_email", cleaners.normalize_email, (" RANI@MAIL.COM ",), "rani@mail.com"),
8     ("normalize_email invalid", cleaners.normalize_email, ("rani-at-mail.com"), None),
9     ("parse_non_negative_float", cleaners.parse_non_negative_float, ("225000.50"), 225000.5),
10    ("parse_positive_int", cleaners.parse_positive_int, ("2"), 2),
11    ("parse_positive_int decimal", cleaners.parse_positive_int, (2.5), None),
12 ]
13
14 for label, fn, args, expected in helper_cases:
15     review_case(label, fn, args, expected)
```

The output shows the results of the cleaning process:

```
normalize_text
actual: 'Rani'
expected: 'Rani'
status: PASS

normalize_text empty
actual: None
expected: None
status: PASS
```

5. Summary – Result data processing(user) yang sesuai standar data quality di kontrak. Diperoleh informasi total data, data bersih/benar, data reject dengan cocokan sesuai kontrak



The screenshot shows a Jupyter Notebook titled 'Pra-bootcam1.ipynb'. The left sidebar displays a file explorer with a folder named 'sample_data' containing 'Data_Quality_Review.ipynb', 'products.json', 'transactions.json', and 'users.json'. The main area shows a code cell with the following Python code:

```
1 importlib.reload(cleaners)
2 try:
3     users_clean, users_rejected, user_decisions = cleaners.clean_users(users_raw)
4     print(f"users: raw={len(users_raw)} clean={len(users_clean)} rejected={len(users_rejected)}")
5     print("clean sample:", users_clean[:2])
6     print("rejected:", users_rejected)
7     print("decision sample:", user_decisions[:5])
8 except Exception as exc:
9     print(f"PERLU DITINJAU: {type(exc).__name__}: {exc}")
```

The output of the code cell is as follows:

```
... users: raw=16 clean=14 rejected=2
clean sample: [{"user_id": "U001", "name": "Rani Putri", "email": "rani@mail.com", "city": "Bandung"}, {"user_id": "U002",
rejected: [{"raw_index": 14, "row": {"user_id": "", "name": "No ID", "email": "noide@mail.com", "city": "Bandung"}, "reason":
decision sample: [{"entity": "users", "id": "U001", "field": "name", "raw": "Rani Putri", "action": "FIX", "clean": "f
```

6. Summary – Result data processing(produk & transaksi) : Diperoleh informasi total data, data bersih/benar, data reject dengan cocokan sesuai kontrak (data quality standar)



The screenshot shows the same Jupyter Notebook interface. The code cell now contains the following Python code:

```
1 importlib.reload(cleaners)
2 for name, rows, fn in [
3     ("products", products_raw, cleaners.clean_products),
4     ("transactions", transactions_raw, cleaners.clean_transactions),
5 ]:
6
7     try:
8         clean, rejected, decisions = fn(rows)
9         print(f"{name}: raw={len(rows)} clean={len(clean)} rejected={len(rejected)} decisions={len(decisions)}")
10    except Exception as exc:
11        print(f"{name}: PERLU DITINJAU - {type(exc).__name__}: {exc}")
```

The output of the code cell is as follows:

```
... products: raw=12 clean=10 rejected=2 decisions=8
transactions: raw=15 clean=12 rejected=3 decisions=6
```

7. Recommendation – Result proccessing sudah divalidasi (true). Data-data yang valid maka layak untuk disubmit ke user. Kesimpulan persetujuan data layak/siap di review user



The screenshot displays a Jupyter Notebook titled "Pra-bootcam1.ipynb". The left sidebar shows a file explorer with a tree view containing folders like ".config", "drive", ".Trash-O", ".shortcut-targets-by-id", "MyDrive", and "sample_data", along with files "Data_Quality_Review.ipynb", "products.json", "transactions.json", and "users.json". The main area contains a code cell with the following Python code:

```
1 for command in [
2     [sys.executable, "run_pipeline.py"],
3     [sys.executable, "validate_delivery.py"],
4 ]:
5     result = subprocess.run(command, text=True, capture_output=True)
6     print("$", ". ".join(command))
7     print(result.stdout)
8     if result.stderr:
9         print(result.stderr)
```

The output of the code cell shows the execution of two commands:

```
... $ /usr/bin/python3 run_pipeline.py
[OK] users: raw=16 clean=14 rejected=2
[OK] products: raw=12 clean=10 rejected=2
[OK] transactions: raw=15 clean=12 rejected=3

$ /usr/bin/python3 validate_delivery.py
PASS users {'clean count': True, 'rejected count': True, 'conservation': True, 'IDs present': True, 'IDs unique': True, 'r
PASS products {'clean count': True, 'rejected count': True, 'conservation': True, 'IDs present': True, 'IDs unique': True,
PASS transactions {'clean count': True, 'rejected count': True, 'conservation': True, 'IDs present': True, 'IDs unique': T
PASS user semantics {'U001 exists': True, 'U001 normalized email': True, 'U003 invalid email null': True}
PASS transaction quantities
PASS decision actions/schema {'NULL', 'DROP', 'FIX'}
```

The output concludes with "READY FOR REVIEW".

Dari sini ada ilmu tentang grand strategi flow proses olah data, SOP, dan parameter pengukuran kinerja dan kualitas tiap proses dan data oleh Data Engineer